

终身学习学分银行平台·积分管理系统

1. 架构设计方案（模块化单体·升级版）

一句话定位：一个 SpringBoot 应用一键部署，内部按领域清晰分模块，用 Redis / Kafka / Spark 把"实时 + 异步 + 离线"三条链路讲透，既稳又能秀。

2. 一、方案定位与选型理由

本系统核心是积分与学分的交易（发放、消耗、转换），业务上最看重的是一致性，而不是超大并发。基于这个特点，我们采用 模块化单体（Modular Monolith），而不是传统紧耦合单体，也不是微服务。

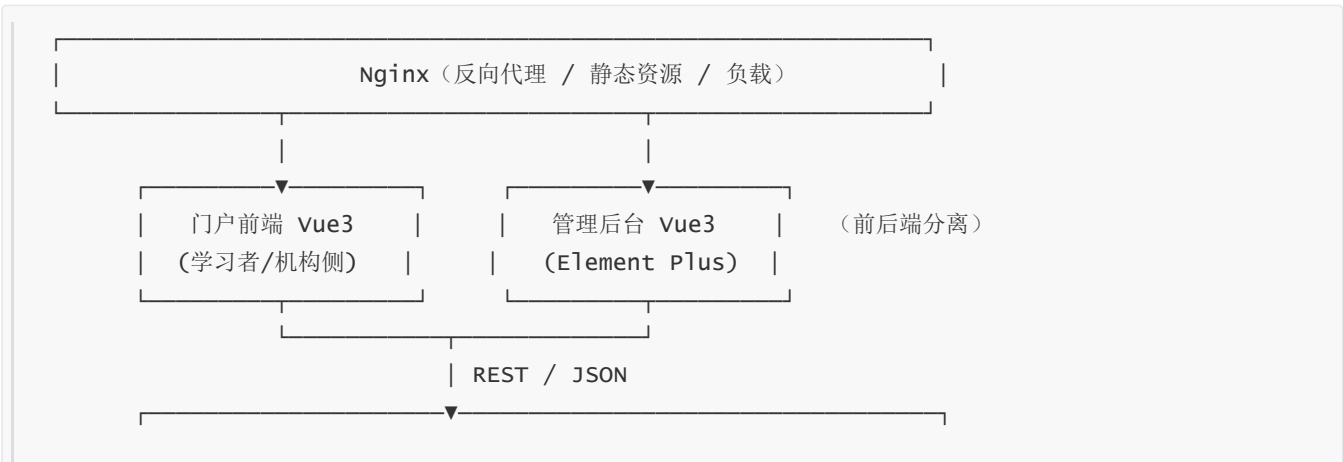
对比项	传统单体	模块化单体（本方案）	微服务
部署	一个包	一个包，Docker 一键起	多服务，需注册中心
交易一致性	本地事务，简单	本地事务，简单	分布式事务，复杂易错
代码耦合	高	低（模块接口隔离）	低
运维/联调成本	低	低	高
架构演进能力	差	好（预留拆分边界）	——
答辩风险	显得简单	可控，能讲权衡	演示易翻车

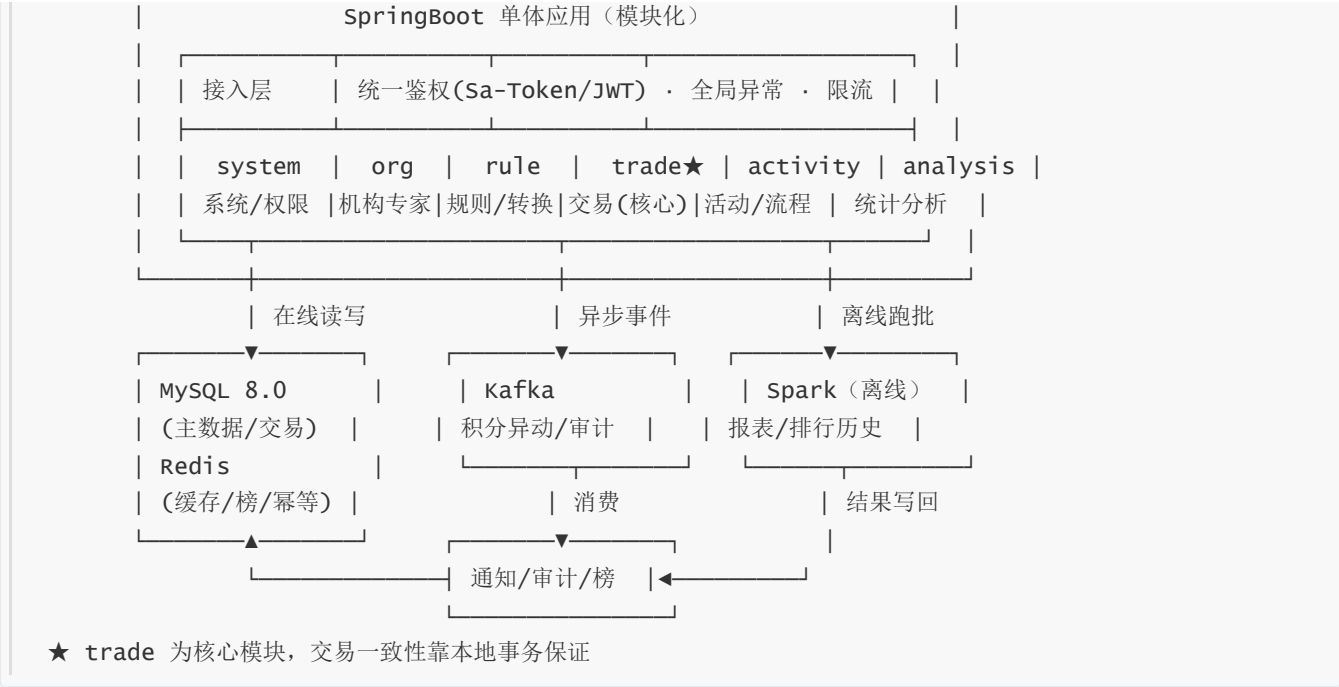
Table 1

为什么不上微服务：一次积分交易涉及「扣积分 + 加学分 + 写流水 + 发通知」，微服务下会变成跨服务的分布式事务，需要 Seata/Saga 补偿，对小组是净负担，且演示现场任一服务没起来整条链路就挂。模块化单体用本地事务天然规避了这个问题。

升级点：在保持"一个应用"的前提下，把代码按领域拆成 Maven 多模块，模块间只通过接口调用，数据库表按模块归属。答辩时可以讲："我们预留了微服务演进边界，未来只需把交易模块独立部署即可" —— 这句话本身就是成熟的架构表达。

3. 二、总体架构图





Fence 1

三条链路一句话概括（这是全篇的灵魂，答辩必讲）：

- **在线链路**：交易请求 → 本地事务写 MySQL → 强一致
- **异步链路**：交易成功 → Kafka 事件 → 通知/审计/实时榜 → 最终一致，解耦
- **离线链路**：Spark 每日跑批 → 统计报表/历史榜 → 读写分离，扛住大数据量分析

4. 三、技术选型

层面	选型	说明
后端	SpringBoot 3.x + MyBatis-Plus	Maven 多模块
前端	Vue3 + Element Plus + Vite	门户端 + 管理端两套
鉴权	Sa-Token（或JWT）	统一登录、RBAC 权限
数据库	MySQL 8.0	主数据、交易流水
缓存	Redis 7 + Redisson	规则缓存 / 排行榜 / 幂等 / 分布式锁 / Session
消息	Kafka	积分异动通知、审计日志、ODS 同步
大数据	Spark（离线批处理）	统计报表、历史排行，定时触发
网关/代理	Nginx	反向代理、静态资源、限流
部署	Docker Compose	MySQL + Redis + Kafka + App 一键启动
定时任务	XXL-Job 或 Spring @Scheduled	本地消息补偿、Spark 任务触发
工具链	Git、PowerDesigner、Navicat、LoadRunner	建模/压测

Table 2

5. 四、模块划分（Maven 多模块）

credit-bank （父工程，聚合 + 依赖版本管理）

├─ credit-common

├─ credit-system

├─ credit-org

├─ credit-rule

├─ credit-trade ★

├─ credit-activity

├─ credit-analysis

└─ credit-boot

公共：工具、统一返回体、异常、常量、基础实体、Redis/Kafka 封装

系统域：管理用户、RBAC 权限、数据字典、系统日志

组织域：机构管理、专家管理、项目管理

规则域：积分规则、转换规则、认证标准

交易域（核心）：账户、交易、余额、本地消息表

活动域：平台活动、业务流程（流程引擎）

分析域：Spark 任务、统计报表、排行榜聚合

启动模块：聚合所有 module，唯一可运行 jar

Fence 2

功能需求 → 模块 映射（把老师给的 10 个功能点全覆盖）：

原始功能	归属模块
积分规则管理	credit-rule
转换规则管理（学分↔积分）	credit-rule
认证标准管理	credit-rule
业务流程管理	credit-activity
平台活动管理	credit-activity
交易管理	credit-trade ★
项目管理	credit-org
专家管理	credit-org
管理用户管理	credit-system
机构管理	credit-org

Table 3

模块间调用规则（这是"低耦合"的关键，答辩要强调）：模块之间只依赖对方暴露的 api 接口，禁止直接访问对方的 Mapper/表。例如 trade 需要规则时，调用 RuleFacade.getPointsRule(code)，而不是自己去查 t_points_rule。这样每个模块都能独立测试，未来任意模块可平滑拆成微服务。

6. 五、数据库设计与表归属

按模块划分表，表名加模块前缀，逻辑清晰、便于将来分库。

模块	主要数据表	说明
credit-system	t_admin_user t_role t_permission t_user_role t_role_permission t_dict t_sys_log	管理用户 + RBAC + 字典 + 日志

模块	主要数据表	说明
credit-org	t_org t_expert t_project t_project_expert	机构 / 专家 / 项目
credit-rule	t_points_rule t_convert_rule t_cert_standard	积分规则 / 转换规则 / 认证标准
credit-trade ★	t_account (积分+学分账户) t_trade_record (交易流水) t_trade_detail t_local_message (本地消息表)	核心, 本地事务保证一致
credit-activity	t_activity t_activity_join t_process t_process_node t_process_instance	活动 + 报名 + 流程
credit-analysis	t_stat_points_daily t_stat_leaderboard t_report	Spark 产出的结果表

Table 4

核心表字段要点（写文档时展开画 PDM）：

- **t_account**：user_id, points_balance, credit_balance, version(乐观锁) —— 靠 version 防并发超扣。
- **t_trade_record**：request_id(幂等键,唯一索引), user_id, type(发放/消耗/转换), points, credit, status, create_time —— 每笔交易一条流水，可追溯。
- **t_local_message**：msg_id, topic, payload, status(待发送/已发送), retry_count —— 保证"交易成功→通知一定发出"的最终一致性（见亮点 2）。

7. 六、核心亮点设计（重点，全是加分项）

7.1 亮点 1：交易一致性 —— 本地事务 + 乐观锁 + 幂等

积分交易是系统命门。我们把「扣积分 / 加学分 / 写流水 / 写本地消息」放在同一个本地事务里，一次提交，要么全成功要么全回滚。

- **防并发超扣**：t_account 用 version 乐观锁，UPDATE ... SET balance=balance-#{n}, version=version+1 WHERE id=#{id} AND version=#{v} AND balance>=#{n}，影响行数为 0 则重试或失败。
- **防重复提交**：前端每次交易带唯一 requestId，进入时先 Redis SETNX idem:trade:{requestId}，命中直接返回，t_trade_record 的 request_id 建唯一索引兜底。

这是单体相对微服务的天然优势——一句"本地事务保证强一致"就避开了微服务需要 Seata 的所有坑。

7.2 亮点 2：Kafka 异步解耦 + 本地消息表（最终一致性）

交易主流程只做核心写库，其余副作用（站内信、短信、审计日志、给 Spark 同步 ODS 数据）全部异步化，通过 Kafka 解耦。

难点在于"消息与事务不一致"（事务回滚了消息却发出去 / 事务成功了消息丢了）。我们用 本地消息表方案 解决：

1. 事务内：核心写库 + 向 t_local_message 插一条 status=待发送 的记录（同一事务，一起提交）。

- 2. 事务提交后：投递 Kafka，成功则更新 `status=已发送`。
- 3. 定时任务补偿：每隔 N 秒扫描 `status=待发送` 且超时的消息重发。

这样保证「交易成功 → 通知最终一定发出」，是教科书级的可靠消息最终一致性，答辩含金量很高。

Kafka Topic 设计：`points-changed`（积分异动）、`audit-log`（审计）、`ods-sync`（同步给 Spark 的宽表）。

7.3 亮点 3：Redis 一物多用（把缓存讲出层次感）

不是简单"加个缓存"，而是四个场景各司其职：

场景	Redis 结构	用途
规则缓存	String / Hash	积分/转换规则读多写少， <code>rule:points:{code}</code> ，规则变更时删缓存
实时排行榜	ZSet	<code>rank:points:global</code> ，交易成功后 <code>ZINCRBY</code> 实时更新， <code>ZREVRANGE</code> 秒出榜
交易幂等	String(SETNX)	<code>idem:trade:{requestId}</code> ，防重复提交
分布式锁	Redisson	活动名额秒抢场景 <code>lock:activity:{id}</code> ，防超发
会话共享	Spring Session	登录态存 Redis，为将来多实例部署铺路

Table 5

排行榜是很亮的点：Redis ZSet 做**实时榜**（毫秒级），Spark 做**历史榜/周月榜**（离线），二者对比正好体现"在线用 Redis、离线用 Spark"的分工。

7.4 亮点 4：Spark 离线分析（大数据能力落地）

在线交易走 MySQL，繁重的统计分析交给 Spark **每日凌晨跑批**，避免拖垮在线库（读写分离思想）：

- 每日积分发放 / 消耗总量、趋势
- 学分↔积分转换趋势分析
- 机构活跃度排名、专家参与度
- TOP100 学习者历史榜
- 结果写回 `t_stat_*` 结果表，前端"统计分析"页直接读结果表，秒开

触发方式：XXL-Job 定时 → 调 Spark 任务 → 从 `ods-sync` 落地的宽表读数 → 聚合 → 写 MySQL 结果表。

7.5 亮点 5：模块化 + 统一网关能力 + 演进边界

- **统一鉴权/限流/异常**：接入层用 Sa-Token 做 RBAC，全局异常处理器统一返回体，接口限流。
- **接口隔离**：模块间只调 `Facade` 接口，可独立单测。
- **演进能力**：预留微服务拆分边界，交易模块可最先独立。这句话能体现小组"懂架构权衡"，比硬拆微服务更显成熟。

8. 七、核心业务流程：一次积分交易全链路（答辩主打）

以"学习者完成课程 → 发放积分 → 部分积分转学分"为例：



Fence 3

这一张图把 MySQL / Redis / Kafka / Spark 四个技术点全部串起来了，而且每一步都有合理理由，是全场最能打的一页。

9. 八、部署方案（Docker Compose 一键启动）

```
# docker-compose.yml 结构示意
services:
  mysql:      # MySQL 8.0, 挂载初始化 SQL
  redis:      # Redis 7
  kafka:      # kafka (含 zookeeper 或 KRaft 模式)
  app:        # credit-boot 应用镜像, 依赖上面三者
  nginx:      # 反向代理 + 前端静态资源
# 一条命令: docker compose up -d 全部拉起
```

Fence 4

Spark 可作为独立容器或本地 standalone，由定时任务调度，不参与在线部署链路，避免拖累主流程。

10. 九、小组分工建议（按模块认领，天然并行）

成员	负责	技术亮点归属
A（组长/架构）	credit-trade 交易核心 + 本地消息表 + 整体集成	亮点 1、2
B	credit-rule 规则/转换/认证 + Redis 缓存	亮点 3（缓存）
C	credit-org + credit-system + Sa-Token 权限	亮点 5
D	credit-activity 活动/流程 + 前端门户	流程引擎
E	credit-analysis Spark 跑批 + 排行榜 + 统计页	亮点 3（榜）、4

Table 6

模块边界清晰 → 各自开发互不阻塞 → 最后由 A 集成，这本身就是"工程能力"的体现。

11. 十、答辩亮点话术（可直接背）

- 1. **架构选型**："我们评估过微服务，但积分交易的强一致需求会引入分布式事务，成本高、风险大。所以选**模块化单体**——用本地事务保证一致性，同时按领域拆模块、接口隔离，**预留了微服务演进边界**。"
- 2. **三条链路**："系统有三条链路：**在线**用 MySQL 保强一致，**异步**用 Kafka 解耦通知与审计，**离线**用 Spark 跑统计分析，实现读写分离。"
- 3. **可靠消息**："为了保证'交易成功、通知一定送达'，我们用**本地消息表 + 定时补偿**实现最终一致性，而不是简单地发完消息就不管。"
- 4. **排行榜双榜**："实时榜用 **Redis ZSet** 毫秒响应，历史榜用 **Spark** 离线聚合，在线离线各司其职。"
- 5. **并发安全**："账户用**乐观锁**防超扣，交易用 **Redis 幂等键 + 唯一索引**双重防重复提交。"

建议交付顺序：先搭 credit-common + credit-boot 骨架 → 打通登录/权限 → 做 credit-trade 交易主流程（先跑通本地事务）→ 接 Redis 缓存与幂等 → 接 Kafka 异步通知 → 最后接 Spark 统计。**每一步都能独立演示**，避免最后一起联调翻车。